

Server-Side Request Forgery (SSRF)

Objetivos de aprendizaje

Demostrar quién realiza una petición, separar validación de resolución y conexión, diagnosticar filtros de destino y reconocer cuándo la salida revela el cliente HTTP subyacente.

Prerrequisitos

URL, DNS, loopback, redirects, parsers, callbacks controlados y método de hipótesis.

Fundamentos técnicos

SSRF permite que una aplicación server-side solicite una ubicación no prevista. Una URL reflejada, un redirect o una carga del navegador no bastan. La primitiva mínima es una conexión atribuible al servidor hacia un destino controlado.

Modelo mental

URL original -> parser de validación -> DNS/IP/política -> cliente HTTP
-> redirect -> nueva resolución/política -> conexión -> canal de respuesta

Validar solo la representación original no garantiza el destino efectivo. La política debe considerar esquema, host normalizado, IP resuelta, puerto y redirects.

Superficie de ataque

Importadores, webhooks, avatares remotos, previews, renderizadores PDF, validadores de enlaces, feeds, analítica y formatos que referencian recursos externos.

Cómo identificarla

- Callback único recibido al ejecutar la función, aun sin navegador.
- IP origen y User-Agent compatibles con infraestructura server-side.
- Diferencias reproducibles entre destino que responde, cerrado e inexistente.
- Contenido o errores del destino aparecen en la respuesta.

Preguntas que debo hacerme

1. ¿Quién inicia realmente la conexión?
2. ¿Qué forma valida la aplicación y qué forma consume el cliente?
3. ¿Cuándo resuelve DNS y qué IP conecta?
4. ¿Sigue redirects y revalida cada salto?
5. ¿Qué protocolos soporta realmente el cliente detectado?

Prueba mínima

Usa un listener HTTP propio del laboratorio con token único. Guarda la URL sin abrir vistas que carguen recursos y correlaciona tiempo, IP, cabeceras y endpoint. Un control apunta a un puerto cerrado del mismo host.

Construcción progresiva del payload

1. Confirmar callback externo.
2. Distinguir visible de blind.
3. Comparar loopback permitido, bloqueado y destino inexistente solo dentro del laboratorio.
4. Identificar si el filtro actúa sobre texto, host parseado o IP efectiva.

5. Probar redirect controlado y observar si existe segunda resolución.
6. Solo tras reconocer cliente y configuración, estudiar capacidades adicionales.

Anatomía de los payloads

- Contexto de entrada: formulario POST /import, campo url.
- Sintaxis original: cliente solicita la URL completa.
- Entrada controlada: esquema, autoridad, puerto y path.
- Transformaciones conocidas: form decode, parser URL, resolución DNS.
- Parser final: cliente HTTP server-side.
- Sink: conexión de red.
- Primitiva: callback HTTP a listener propio.
- Payload mínimo: `http://listener.lab/ssrf-7f31`.
- Significado de cada componente: `http` selecciona protocolo; `host` resuelve al listener; `token` correlaciona la prueba.
- Resultado esperado: una petición server-side al token.
- Control negativo: puerto cerrado y token distinto no enviado.
- Restricción observada: destinos loopback escritos como `127.0.0.1` se bloquean.
- Por qué falla la variante básica: la validación rechaza esa representación antes del cliente.
- Hipótesis de adaptación: validación textual distinta de la normalización del cliente.
- Payload adaptado: representación alternativa solo después de comprobar cómo la interpreta ese cliente.
- Por qué debería funcionar: ambas formas podrían resolverse a loopback, pero el filtro compara solo una.
- Evidencia: respuesta local y traza de conexión a la IP efectiva.
- Cuándo no funcionaría: política sobre IP canónica, cliente que no acepta la forma o revalidación correcta.

Variaciones según el contexto

SSRF visible devuelve contenido; blind exige callback o señal temporal robusta. Un redirect solo cambia el destino si el cliente lo sigue. Un protocolo alternativo solo existe si el cliente lo implementa y la política lo permite; conocer su nombre no demuestra soporte.

Filtros y bypasses

Clasifica antes de adaptar: esquema no permitido, host no permitido, IP privada, puerto, DNS, redirect o salida. Prueba una variable cada vez. Las representaciones alternativas de IP dependen del parser; nunca se presentan como universales.

Evidencias de confirmación

Callback atribuible al backend o contenido inequívoco obtenido por él. La presencia de `url=`, un error DNS genérico o una demora aislada son indicios.

Escalado de impacto

Determinar alcance de destinos y puertos con pruebas mínimas, observar identidad o cabeceras añadidas, y documentar si el backend accede a recursos no alcanzables externamente. No explorar terceros ni metadatos cloud fuera de un laboratorio preparado.

Errores frecuentes

- Confundir carga de `<img>` con SSRF.
- Asumir que un redirect siempre se sigue.

- Probar muchas representaciones sin conocer parser.
- Inferir curl solo por una barra parecida.
- Confundir SSRF con inyección de argumentos en un cliente CLI.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
listener recibe callback, localhost no	SSRF confirmada con política interna	comparar IP cerrada y bloqueo explícito
127.0.0.1 bloqueado, forma equivalente funciona	validación textual o normalización diferencial	registrar host parseado e IP conectada
callback solo al abrir navegador	carga client-side	guardar sin render y comparar origen
primer host permitido, redirect interno bloqueado	revalidación tras redirect	redirect externo a externo como control
redirect interno funciona	validación solo del primer salto	capturar ambos requests y DNS
aparece salida de curl	posible CLI o wrapper	opción inocua y firma/versionado, sin asumir splitting
file:// bloqueado	política de esquema o cliente sin soporte	comparar error de parser con esquema inventado

Mitigaciones

Allowlist de destinos necesarios, parser único, resolución y comprobación de IP efectiva, bloqueo de redes no permitidas, revalidación tras cada redirect, límites de protocolo/puerto y egress controlado.

Relación con pentesting y certificaciones

La competencia clave es atribuir la conexión y explicar la discrepancia entre validación y destino, no memorizar escrituras de loopback.

Caso guiado

Caso A — Básico: importador de feed

POST /feeds/import acepta url. Un token único llega al listener durante el POST con User-Agent: FeedWorker/2.1, aunque nadie abre la vista.

Observación: callback correlacionado con el importador. Qué sé: un componente server-side solicita la URL.

Hipótesis: worker directo o cola asíncrona. Experimento: dos tokens con tiempos y un destino cerrado. Predicción: la cola puede retrasar, pero mantiene origen y token; el navegador no participa. Resultado: callbacks desde la red del servidor y error diferente para puerto cerrado. Conclusión: SSRF básica confirmada. Siguiente paso: estudiar política de destinos con marcadores inocuos.

Caso de adaptación

Caso B — Representación, parser y herramienta

http://127.0.0.1/ devuelve Private network access blocked. Una forma abreviada de loopback aceptada por el cliente devuelve HTML local. La respuesta incluye % Total, % Received y % Xferd.

Observación: dos textos acaban potencialmente en el mismo destino y aparece una firma compatible con curl. Qué sé: el filtro no trató ambas formas igual; todavía no sé si la salida procede de curl real ni si existe splitting. Hipótesis: H1, regex textual + CLI curl; H2, biblioteca que reproduce el formato; H3, destinos realmente distintos. Experimento: registrar IP conectada, comparar User-Agent, provocar un error de opción inocuo solo si existe evidencia de

argumentos separados y consultar la versión expuesta por el laboratorio. Predicción: H1 muestra misma IP y errores propios de curl; H2 no acepta opciones; H3 conecta distinto. Resultado: misma IP y error oficial de curl al recibir una opción separada por el wrapper. Conclusión: SSRF con validación textual y un segundo riesgo potencial de argumentos. Siguiente paso: estudiar documentación de esa versión; no saltar a un protocolo o fichero final.

Caso C — Transferencia: documento mensual

Una función genera PDF desde una plantilla que contiene un logo remoto. El título no nombra la técnica. Al usar un token de imagen, el callback ocurre durante POST /reports/render, desde el renderer.

Resolución: entrada en HTML -> parser del renderer -> cliente de recursos -> conexión. La primitiva es la misma aunque no exista parámetro url. El siguiente experimento compara recurso externo, relativo y bloqueado, sin apuntar a servicios sensibles.

Caso D — Falso positivo: avatar remoto

Guardar avatar_url no produce tráfico; abrir el perfil en Firefox sí, con IP y User-Agent del alumno.

Observación: hay callback. Hipótesis: backend o navegador. Experimento: guardar sin renderizar, solicitar JSON crudo y comparar origen. Resultado: solo el navegador conecta. Conclusión: petición client-side; SSRF descartada. Siguiente paso: evaluar controles de contenido del navegador solo si entra en alcance.

Ejercicios

1. Diseña un control para separar DNS server-side de HTTP server-side.
2. Una URL permitida redirige dos veces: indica qué registrar en cada salto.
3. Explica qué evidencia necesitas antes de probar una opción de curl.
4. Distingue SSRF, open redirect y carga client-side en tres trazas dadas.

Resumen

SSRF se confirma atribuyendo una conexión al servidor. Los bypasses se explican mediante parser, resolución y política; las capacidades del cliente se estudian después de identificarlo.

Chuleta operativa

1. Token único y listener.
2. Origen, tiempo y User-Agent.
3. URL original y parseada.
4. DNS e IP efectiva.
5. Redirect y segunda validación.
6. Cliente/versionado con evidencia.
7. Control cerrado e inexistente.

Referencias

- [PortSwigger: SSRF](#)
- [RFC 3986](#)
- [curl manual](#)
- [Concepto interno \(ssrf\)](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Práctica: [cuaderno](#).